Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod) 5

What you get . . . 5

What this delivers . . . 5

The deployment model . . . 6

Outcome . . . 6

1. Solution Overview — Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod) 6

What this solution is . . . 6

Part 1 — OUAF configuration migration (CMA / bundles) . . . 7

Part 2 — Code and web-service (IWS) promotion . . . 7

Part 3 — Environment comparison (parity) . . . 7

The Dev → QA → Prod flow with gates . . . 8

What this is not . . . 8

The deployment model . . . 8

2. Requirements Guide — Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod) 8

2.1 Source and target environments at compatible levels . . . 8

2.2 Migration Assistant Configuration enabled on both environments . . . 9

2.3 Migration Plans for all objects in scope (including custom) . . . 9

2.4 A named Migration Request per promotion scope . . . 9

2.5 File transfer path between environments . . . 9

2.6 Code build / patch artifact and IWS deployment access . . . 10

2.7 Change control, backups, and a rollback point per environment . . . 10

2.8 A pilot promotion scope . . . 10

Checklist . . . 10

Global Requirements — Oracle Utilities C2M / WebLogic Solutions 11

1. Licensed Oracle Utilities C2M + WebLogic entitlement . . . 11

2. Certified platform stack (OS, Java, Database, WebLogic) . . . 11

3. Environment topology (Dev / Test / Prod) & access . . . 11

4. Administrative access & tooling (WLST, shell, SQL) . . . 12

5. Change control, backup & a non-production pilot . . . 12

6. Collaborating teams & support model . . . 12

3. Implementation Guide — Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod) 13

Step 1 — Assess the environments and scope . . . 13

Step 2 — Configure the migration objects . . . 13
Step 3 — Export the configuration bundle from the source . . . 13
Step 4 — Promote the code build . . . 14
Step 5 — Import the bundle into the target . . . 14
Step 6 — Deploy Inbound Web Services on the target . . . 14
Step 7 — Compare environments and confirm parity . . . 14
Step 8 — Validate against the environment gate . . . 15
Step 9 — Pilot Dev → QA, then expand QA → Prod . . . 15
5. Testing Guide — Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod) 15
Test 1 — Source and target are at compatible levels . . . 16
Test 2 — The bundle imported and applied . . . 16
Test 3 — Configuration objects match (no drift) . . . 16
Test 4 — Code build / patch level is identical . . . 16
Test 5 — Inbound Web Services are deployed and current . . . 16
Test 6 — Review caught nothing unexpected . . . 16
Test 7 — Rollback point is verified (Production) . . . 16
Pass/fail gate . . . 16
6. Troubleshooting Guide — Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod) 17
A. An object is missing in the target after import . . . 17
B. The bundle will not export or import . . . 17
C. Inbound Web Services are stale or failing . . . 18
D. Configuration is live but behaves wrong — code is out of step . . . 18
E. The import review showed unexpected changes . . . 18
F. Compare report shows a gap after a "successful" import . . . 18
G. A promotion needs to be rolled back . . . 18
H. Diagnostics quick reference . . . 19
I. When to escalate . . . 19
7. Best Practices — Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod) 19
8. Deliverables Summary — Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod) 20
Documentation . . . 20
Named migration objects . . . 21
Automation scripts (shell / WLST / SQL — not PowerShell) . . . 21
What the buyer receives . . . 21
Outcome . . . 22

Executive Summary — Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod)

Oracle Utilities **Customer to Meter (C2M)** configuration and code have to move from **Development to QA to Production** without drift and without manual mistakes. When promotion is ad-hoc — hand-edited configuration in each environment, code that lags the config, web services deployed inconsistently — what goes live is not what was tested, and the first sign of trouble is a production incident. This solution defines the **migration and code-promotion method**: OUAF configuration migration through the **Configuration Migration Assistant (CMA)** as reusable export/import bundles, code and **Inbound Web Service (IWS)** promotion, and a deliberate **environment-comparison** step — so what you tested is exactly what goes live, pilot-first and rollback-ready.

This is a complete, do-it-today package written for an experienced C2M administrator. Every migration object is named once and reused every release, so you build the promotion once and repeat it across environments and releases instead of improvising each time.

What you get

• A plain-language Solution Overview of the C2M promotion model: OUAF configuration migration (CMA /

bundles), code and IWS promotion, environment comparison, and the Dev → QA → Prod flow with explicit go/no-go gates.

• A Requirements Guide with verify / implement-if-missing for every prerequisite — compatible source/target

levels, Migration Assistant Configuration, Migration Plans for custom objects, a named Migration Request, the file-transfer path, code build and `iwsdeploy` access, and change control with a rollback point.

• An Implementation Guide of precise, ordered steps: configure the migration objects, export the bundle,

promote the code build, import into the target, deploy IWS from the command line, compare environments, and validate parity — Dev → QA first, then QA → Prod inside change control.

• Five automation scripts (**shell / WLST / SQL**, not PowerShell) for read-only environment/config

discovery, parity validation, a source-vs-target compare report, dry-run-by-default remediation, and a Migration Request scope helper.

• Testing and Troubleshooting guides with per-environment pass/fail gates, plus Best Practices for a

repeatable promotion.

What this delivers

A repeatable, gated promotion method — reusable CMA bundles, code and IWS in lockstep, and a compare step that proves parity — so configuration and code move cleanly across environments with no drift and a verified rollback at every hop.

> ***Boundary.*** *Authored from Oracle's official OUAF / C2M documentation. Version-specific object names and commands are cited or flagged "verify for your version" (C2M 25.4 / min v2.9, WebLogic 14c); MOS-gated reference detail and patches are pointed to, not reproduced. All promotion assumes a non-production environment first and change control for Production.*

The deployment model

Assess, then Configure, then Pilot, then Validate, then Expand — prove the promotion Dev → QA on a bounded scope, pass the gate, then repeat QA → Prod with backups and a verified rollback.

Outcome

C2M configuration and code promoted cleanly from Dev to QA to Production — parity confirmed, no drift — so what you tested is exactly what goes live.

1. Solution Overview — Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod)

What this solution is

This is the repeatable method for moving Oracle Utilities **Customer to Meter (C2M)** configuration and code cleanly across environments — **Development** → **QA** → **Production** — without drift and without manual mistakes. It is not a one-off script; it is a defined promotion model with named migration objects you build once and reuse every release, explicit go/no-go gates between environments, and an environment-comparison step that proves what you tested is exactly what goes live.

C2M runs on the **Oracle Utilities Application Framework (OUAF)**, so promotion has three moving parts, each with its own tooling:

1. **OUAF configuration migration** — metadata and configuration objects (business objects, algorithms,

scripts, UI maps, lookups, batch controls, portals, and the rest) move through the framework's **Configuration Migration Assistant (CMA)** as **export/import bundles**.

1. **Code and web-service (IWS) promotion** — custom Java/Groovy extensions and **Inbound Web Services

(IWS)** are promoted through the OUAF product install/patch tooling and the `iwsdeploy` utility, not through CMA.

1. **Environment comparison** — a deliberate parity check confirms the target now matches the source for

the promoted scope, so nothing was dropped or silently overwritten.

> ***Version boundary.*** *CMA, IWS deployment, and the OUAF install tooling are authored from Oracle's official OUAF / C2M product documentation. Version-specific object names, batch codes, and command switches are cited or flagged* ***"verify for your version"*** *(C2M 25.4 / minimum v2.9, WebLogic 14c per the certification matrix). Some CMA reference detail and patches are gated on* ***My***

> ***Oracle Support (MOS)**; where a step needs a MOS note or patch, this solution points to it rather than reproducing gated content. All promotion assumes a **non-production environment first** and change control for production.*

Part 1 — OUAF configuration migration (CMA / bundles)

CMA is the framework's facility for exporting configuration from a source environment and importing it into a target, as a self-contained file. The moving parts are named framework objects you define once:

- **Migration Assistant Configuration** — the system-wide master configuration record that enables and parameterizes CMA on each environment.
- **Migration Plan** — defines *what a record type looks like* to CMA: the maintenance object (MO) and business object to copy, and the related records to pull along with it. You reuse the delivered plans and add plans only for custom objects.
- **Migration Request** — defines *the specific set of data* to include in a given export: which configuration to bundle for this promotion. The Migration Request is your reusable, named unit of promotion — build one per logical scope (for example, one for rate configuration, one for a to-do type set) and reuse it every release.

The export produces a **binary export file** (the "bundle"). That file is transferred from the source's export directory to the target's import directory, then imported. On import, CMA reads the file, compares each object to what already exists in the target, lets you **review the proposed changes**, and then applies them. Two behaviors matter for gating: when a record exists in both source and target, CMA **fully replaces** the target object with the source version; and the import **does not delete** records that exist only in the target. That is why the comparison step (Part 3) is not optional.

Because the export file is just a file, it can be **checked into version control** — which is what makes promotion auditable and repeatable: the same reviewed bundle that passed QA is the artifact you apply to Production.

Part 2 — Code and web-service (IWS) promotion

Configuration is not code. Custom extension code (Java/Groovy, framework hooks) and any file-system artifacts are promoted through the **OUAF product install/patch mechanism** — you deploy the same build into each environment and run the framework's post-install steps — not through CMA.

Inbound Web Services (IWS) are a special case. IWS metadata migrates as configuration through CMA, but the running service must be **deployed** into WebLogic afterward. The online *Inbound Web Service Deployment* portal (Admin → Integration) is **supported only in development / non-production environments**; for QA and Production, deployment is done at the command line with `iwsdeploy[.sh]`, run under the product environment (`splenviron`). After deployment, the *Deployment Status* zone shows each service as current or stale by comparing its service revision against the value captured at deploy time — a built-in parity signal.

Part 3 — Environment comparison (parity)

The final step of every promotion is a **comparison** of source and target for the promoted scope: confirm the configuration objects in the Migration Request now match, confirm the IWS deployment status is

current, and confirm the code build/patch level is identical. Parity confirmed is the evidence that "what you tested is what went live." The included scripts automate the read-only side of this: inventory what is present, validate parity, and produce a compare report.

The Dev → QA → Prod flow with gates

Promotion is always one direction and always gated:

- **Dev → QA.** Author and unit-test in Dev. Export the Migration Request bundle, promote the code build,

deploy IWS, import into QA, then compare. **Gate:** QA parity confirmed and QA functional testing passed before anything targets Production.

- **QA → Prod.** Promote the *same* reviewed bundle and the *same* build to Production inside a change

window, with backups taken first. Import, deploy IWS with `iwsdeploy`, then compare. **Gate:** Production parity confirmed against QA and a rollback point verified.

Every gate has an explicit pass/fail check (see the Testing Guide) and a documented rollback (restore the pre-import backup / re-import the prior bundle).

What this is not

This solution defines and automates the *promotion method*; it does not author your C2M configuration or write your extension code. The included scripts are **read-only where they touch live systems** — discovery, validation, and comparison — plus one remediation helper that is **dry-run by default**. They inventory and prove parity; they do not push configuration or move code on their own.

The deployment model

> *Assess the environments and scope, configure the migration objects (Migration Request, plans, IWS deployment), pilot the promotion Dev → QA on a bounded scope, validate parity against the gate, then expand to QA → Prod. The repeatable way:* ***Assess, then Configure, then Pilot, then Validate, then Expand.***

2. Requirements Guide — Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod)

Verify every requirement before promoting anything. Each entry: Description, Why, Verify, Implement-if-missing. Category-wide prerequisites (entitlement, certified stack, WLST/shell/SQL tooling, change control) are in `global-requirements-oracle.md` and are assumed here.

2.1 Source and target environments at compatible levels

Description: A **source** environment (where the change is authored — Dev) and a **target** environment (QA, then Prod) running the same C2M/OUAF release and, ideally, the same patch level. CMA supports migrating across framework versions, but same-version promotion is the safe default for a release. Why: CMA import compares and replaces objects in the target; a large framework-version gap between source

and target introduces schema and object-definition differences that must be reconciled first. Verify: Record the C2M release, OUAF version, and patch/bundle level on both environments; run `OC2MMIG-Discovery.sh` against each. Confirm they match (or that any gap is intended and understood). Implement-if-missing: Align the target to the source's release/patch level before promoting, or plan the version-upgrade path per Oracle's cross-version CMA guidance (verify for your version on MOS).

2.2 Migration Assistant Configuration enabled on both environments

Description: The system-wide **Migration Assistant Configuration** master record configured on the source and the target so CMA can export and import. Why: CMA will not export or import without its master configuration in place; it defines the export/import directories and framework behavior on each environment. Verify: The Migration Assistant Configuration record exists and its export/import file directories are readable/writable by the product OS account; `OC2MMIG-Validation.sh` reports the CMA-readiness signal. Implement-if-missing: Complete the Migration Assistant Configuration on each environment and confirm the export and import directories exist on the file system with the correct ownership.

2.3 Migration Plans for all objects in scope (including custom)

Description: A **Migration Plan** for every configuration object type being promoted. The delivered plans cover framework and product objects; any **custom** maintenance object/business object needs its own plan. Why: A Migration Request can only bundle object types that have a Migration Plan; a custom object with no plan is silently left out of the promotion — a classic cause of "it worked in QA but is missing in Prod." Verify: For each custom object in scope, confirm a Migration Plan exists that names its MO and business object and pulls the related records. Inventory with `OC2MMIG-Discovery.sh`. Implement-if-missing: Author a Migration Plan for each custom object type (MO + business object + related records) before building the Migration Request.

2.4 A named Migration Request per promotion scope

Description: A reusable **Migration Request** that defines exactly which configuration this promotion bundles — built once per logical scope and reused every release. Why: The Migration Request is the unit of promotion. A stable, named request is what makes releases repeatable and auditable; ad-hoc "select everything" exports are neither. Verify: The Migration Request for the scope exists and lists the intended objects; `OC2MMIG-ScopeHelper.sh` prints the request's declared scope so you can confirm it matches the change. Implement-if-missing: Create a Migration Request scoped to the change (for example, one for rate configuration, one for a to-do/algorithm set) and name it so it is reused, not recreated, each release.

2.5 File transfer path between environments

Description: A controlled way to move the CMA **binary export file** from the source's export directory to the target's import directory (secure copy, a shared staging area, or a version-control checkout). Why: CMA export and import are two steps joined by a file; if the file cannot reach the target's import directory the promotion cannot proceed. Verify: You can copy a test file from the source export directory to the target import directory with the product OS account and correct permissions. Implement-if-missing: Establish the transfer path (scp/sftp or shared mount) and, preferably, a version-control location so every promoted bundle is retained and auditable.

2.6 Code build / patch artifact and IWS deployment access

Description: The promotable **code build or patch** (the same artifact installed into each environment) and command-line access to run `iwsdeploy[.sh]` under the product environment (`splenviron`) on the target for **Inbound Web Services** deployment. Why: Configuration alone is not the whole change; custom code and IWS must be promoted too. IWS deployment to QA/Prod is command-line only — the online deployment portal is supported **only in non-production** environments. Verify: The build/patch artifact is identified and versioned; you can source the product environment and run `iwsdeploy` on the target. `OC2MMIG-Validation.sh` reports the IWS deployment-status signal. Implement-if-missing: Produce a versioned build/patch, stage it for each environment, and confirm shell access to run the OUAF install steps and `iwsdeploy` on QA and Prod.

2.7 Change control, backups, and a rollback point per environment

Description: A change window and approval for the target, a verified **backup** taken before import (database/config export, and note of the current bundle/patch level), and a defined rollback. Why: CMA import **replaces** existing target objects and does not delete extras; a bad promotion must be recoverable. Production import happens inside change control with a proven restore path. Verify: A backup/restore path is confirmed and a change window/approval exists for the target; the current target bundle level is recorded so you can re-import it if needed. Implement-if-missing: Establish the backup/restore procedure and change-control approval before importing into QA or Prod (see `global-requirements-oracle.md` §5).

2.8 A pilot promotion scope

Description: A bounded, well-understood first promotion — a single Migration Request covering one object family — to prove the Dev → QA path before promoting the full release or touching Production. Why: The method is validated on a pilot scope first so the migration objects, transfer path, IWS deployment, and comparison are confirmed end to end cheaply. Verify: A pilot Migration Request and its owners are identified; `OC2MMIG-ScopeHelper.sh` resolves its scope. Implement-if-missing: Choose a small, representative configuration set as the first promotion.

Checklist

- Source/target at compatible C2M/OUAF levels — `OC2MMIG-Discovery.sh`
- Migration Assistant Configuration enabled on both — `OC2MMIG-Validation.sh`
- Migration Plans exist for all in-scope objects (incl. custom) — `OC2MMIG-Discovery.sh`
- Named Migration Request per scope — `OC2MMIG-ScopeHelper.sh`
- File transfer path source export dir → target import dir
- Code build/patch artifact + `iwsdeploy` access — `OC2MMIG-Validation.sh`
- Backups, change control, and rollback point per environment
- Pilot promotion scope identified — `OC2MMIG-ScopeHelper.sh`

Global Requirements — Oracle Utilities C2M / WebLogic Solutions

These requirements apply to **every** solution in the **Oracle** category (Oracle Utilities Customer to Meter and its Oracle WebLogic Server middleware). Verify each before starting a solution's steps.

> ***Vendor & version boundary (applies to every Oracle solution).*** *Oracle Utilities* ***Customer to Meter (C2M)****,* ***Customer Care & Billing (CC&B)****,* ***Meter Data Management (MDM)****, and* ***Oracle WebLogic Server*** *are authored from Oracle's official product documentation and the Oracle Utilities Application Framework (OUAF) install/admin guides.* ***Version-specific details*** *(C2M 25.4 / minimum v2.9, WebLogic 14c) are* ***cited or flagged "verify for your version"*** *— never invented. Some Oracle documentation and patches are entitlement-gated on* ***My Oracle Support (MOS)****; where a step requires a MOS note, patch number, or download, the solution points to it rather than reproducing gated content. All work assumes a* ***non-production environment first*** *and change control for production.*

1. Licensed Oracle Utilities C2M + WebLogic entitlement

• **Description:** A licensed Oracle Utilities **Customer to Meter** (and the underlying OUAF) plus the supported **Oracle WebLogic Server 14c** and database (Oracle Database) at the versions in scope.

• **Why required:** Installation media, patches, and support all flow from the entitlement.

• **How to verify:** Confirm the license and access to **My Oracle Support**; check the certified version matrix for C2M ↔ WebLogic ↔ Database ↔ Java.

• **How to implement if missing:** Obtain licensing/entitlement and MOS access before proceeding.

2. Certified platform stack (OS, Java, Database, WebLogic)

• **Description:** A host OS, JDK, Oracle Database, and WebLogic version combination **certified** for the target C2M release (per Oracle's certification/compatibility matrix).

• **Why required:** C2M/OUAF only supports certified combinations; uncertified stacks cause install and patch failures and lose supportability.

• **How to verify:** Compare the environment against the C2M release's certification matrix (MOS).

• **How to implement if missing:** Align OS/JDK/DB/WebLogic to a certified combination first.

3. Environment topology (Dev / Test / Prod) & access

• **Description:** Defined **Development, Test, and Production** environments (and any QA/UAT), with OS accounts (e.g. the OUAF/`splunk`-style product user), WebLogic admin credentials, and DB schema access appropriate to the task.

• **Why required:** C2M is deployed and promoted across environments; each solution assumes the target

environment exists and you can reach WebLogic, the file system, and the database.

• **How to verify:** Confirm you can log in to the OS product account, the WebLogic Administration Console / WLST, and the C2M database schema.

• **How to implement if missing:** Stand up / obtain access to the environment before starting.

4. Administrative access & tooling (WLST, shell, SQL)

• **Description:** Administrative access and the standard toolchain: **WLST (Jython)** for WebLogic, the OS **shell** (bash/ksh) for OUAF utilities (e.g. `splmanager`, `configureEnv`, `initialSetup`, batch/threadpool scripts), and **SQL*Plus / SQLcl** for the database.

• **Why required:** C2M/WebLogic administration is performed through these interfaces, not a GUI-only workflow; the solutions' automation is authored as WLST/shell/SQL.

• **How to verify:** Confirm WLST connects to the domain, shell access to the product home, and SQL connectivity to the schema.

• **How to implement if missing:** Provision the access and install the client tooling.

5. Change control, backup & a non-production pilot

• **Description:** A **change-control** process for production changes, verified **backups** (domain config, OUAF product/config, database) before disruptive actions (install, patch, upgrade, migration, tuning), and a **non-production environment** to validate on first.

• **Why required:** Every solution follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with a documented rollback; C2M patches/upgrades and WebLogic changes are high-impact.

• **How to verify:** Confirm a backup/restore path and a change window/approval for production.

• **How to implement if missing:** Establish backups and change control before touching production.

6. Collaborating teams & support model

• **Description:** Engagement with the **infrastructure, DBA, network, application-development, and release/test **teams, and (post go-live) the** hypercare / on-call** support model and monitoring stack (e.g. Dynatrace) the environment uses.

• **Why required:** C2M administration is cross-functional; issues span middleware, database, network, and application layers, and production support runs on a defined escalation and monitoring model.

• **How to verify:** Confirm the named contacts, escalation path, and monitoring tooling.

• **How to implement if missing:** Identify the teams and the support/monitoring model before go-live.

*The deployment model across every Oracle solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **WLST (Jython), shell, and SQL** scripts — not PowerShell.*

3. Implementation Guide — Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod)

Follow these steps in order. This guide promotes C2M configuration and code across environments: export/import the OUAF configuration bundle, promote the code build, deploy Inbound Web Services, compare environments, and validate parity — Dev → QA first, then QA → Prod, each behind an explicit gate. Object names are fixed; build the Migration Request once and reuse it every release.

> *Model: prove the promotion on a bounded scope Dev → QA, validate parity against the gate, then repeat QA → Prod inside change control. The repeatable way:* ***Assess, then Configure, then Pilot, then Validate, then Expand.***
>
> ***Direction and safety.*** *Promotion is always source → target and one-way. Always take a backup of the target before import (CMA replaces existing objects and does not delete extras). Production is always touched inside a change window with a verified rollback point.*

Step 1 — Assess the environments and scope

1. Run `OC2MMIG-Discovery.sh` against the **source** (Dev) and each **target** (QA, Prod) to inventory the

C2M/OUAF release and patch level, CMA readiness, and the Migration Plans/Migration Requests present.

1. Confirm source and target are at compatible framework levels (Requirement 2.1). Record any gap; do not

proceed past a version mismatch you have not planned for.

1. Define the promotion scope in writing: which configuration objects, which custom objects, which code build/patch, and which Inbound Web Services are changing.

Step 2 — Configure the migration objects

1. Confirm the **Migration Assistant Configuration** master record is enabled on the source and each target, with valid export/import directories (Requirement 2.2).

1. For every configuration object type in scope — including **custom** MOs/business objects — confirm a **Migration Plan** exists (delivered plan reused, or a custom plan authored). A missing plan means the object is silently excluded from the bundle.

1. Create or confirm the named **Migration Request** for this scope. Build it once and reuse it every release; verify its declared scope with `OC2MMIG-ScopeHelper.sh`.

Step 3 — Export the configuration bundle from the source

1. On the source (Dev), run the CMA export for the Migration Request. This produces a **binary export file** (the bundle) in the export directory.

1. Retain the bundle in **version control** (or the controlled staging location) so the exact reviewed

artifact is the one promoted onward — this is what makes the release auditable and repeatable.

1. Record the bundle name/version alongside the code build/patch version and the IWS list; these three travel together as one release.

Step 4 — Promote the code build

1. Install the same versioned **code build / patch** into the target that was tested in the source, using the OUAF product install/patch steps. Configuration migrates by bundle; code migrates by installed build — keep them in lockstep.

1. Run the framework post-install steps required by the build (verify for your version), so the target's file-system artifacts and framework hooks match the source.

Step 5 — Import the bundle into the target

1. Transfer the binary export file from the source export directory to the **target import directory** (Requirement 2.5).

1. Back up the target first (Requirement 2.7): database/config export and a note of the current bundle level, so you can re-import the prior bundle if needed.

1. Run the CMA import. CMA reads the file, compares each object to the target, and presents the **proposed changes** for review. Review them — confirm the objects match the intended scope — before applying.

1. Apply the import. Remember: matching objects are **fully replaced** from the source; target-only objects are **not deleted**. Anything unexpected in the review is your signal to stop and reconcile.

Step 6 — Deploy Inbound Web Services on the target

1. IWS *metadata* arrived with the bundle, but the running service must be deployed. On the target, source the product environment (`splenviron`) and run `iwsdeploy[.sh]` to deploy the in-scope Inbound Web Services.

1. Use the **command line** for QA and Production — the online *Inbound Web Service Deployment* portal (Admin → Integration) is supported **only in non-production** environments.

1. Confirm the *Deployment Status* zone shows each in-scope service as **current** (its service revision matches the deployed value); a stale status means the service still needs deployment.

Step 7 — Compare environments and confirm parity

1. Run `OC2MMIG-Report.sh` to produce a source-vs-target **compare report** for the promoted scope: configuration objects present and matching, IWS deployment status current, and code build/patch level identical.

1. Run `OC2MMIG-Validation.sh` on the target to confirm the parity signals (CMA import applied, IWS

current, build level matched) with no gaps.

1. If the compare report shows a gap, remediate the *cause* — re-run the missing part (re-import the

bundle, re-deploy an IWS, or re-apply the build) — then re-compare. Use `OC2MMIG-Remediation.sh` (dry-run by default) to preview any corrective action before applying it.

Step 8 — Validate against the environment gate

1. Complete the Testing Guide for this target. The promotion passes the gate only when the compare report

shows **parity for the full scope**, IWS deployment status is current, and the code build level matches the source.

1. For **QA**, also confirm functional testing passed on the promoted change before promoting onward.

2. For **Production**, confirm the backup/rollback point is verified and the change window/approval is in

place before and after the import.

Step 9 — Pilot Dev → QA, then expand QA → Prod

1. **Pilot:** run Steps 3–8 for the bounded pilot Migration Request Dev → QA. Passing the QA gate proves

the migration objects, transfer path, IWS deployment, and comparison all work end to end.

1. **Expand:** promote the full release Dev → QA, pass the gate, then promote the **same reviewed bundle

and same build** QA → Prod inside change control. Re-run `OC2MMIG-Report.sh` and `OC2MMIG-Validation.sh` for Production and pass the Production gate.

1. Keep the Migration Request, the bundle-in-version-control practice, and the compare step for every

future release — the method is now repeatable.

Order of operations

```
Assess -> Configure migration objects -> Export bundle -> Promote code build
-> Import bundle -> Deploy IWS -> Compare/parity -> Validate gate -> Pilot
Dev→QA -> Expand QA→Prod
```

5. Testing Guide — Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod)

Run after each promotion (Implementation Steps 5–8), for QA first and then again for Production. Each test confirms one part of the promotion landed and that source and target are at parity. The scripts that touch live systems are read-only; the remediation helper is dry-run by default.

Test 1 — Source and target are at compatible levels

Do: Run `OC2MMIG-Discovery.sh` against source and target. Expected: Both report the same C2M/OUAF release and patch level (or an intended, understood gap); no unplanned version mismatch.

Test 2 — The bundle imported and applied

Do: On the target, review the CMA import result for the Migration Request, then run `OC2MMIG-Validation.sh`. Expected: The import shows applied with no errors; validation reports the CMA-import signal present for the promoted scope.

Test 3 — Configuration objects match (no drift)

Do: Run `OC2MMIG-Report.sh` to compare the promoted configuration objects between source and target. Expected: Every object in the Migration Request is present in the target and matches the source; the compare report shows no missing or mismatched objects for the scope.

Test 4 — Code build / patch level is identical

Do: Run `OC2MMIG-Discovery.sh` on source and target and compare the build/patch level. Expected: The target's code build/patch level equals the source's; configuration and code are in lockstep.

Test 5 — Inbound Web Services are deployed and current

Do: On the target, check the *Deployment Status* zone for the in-scope services; run `OC2MMIG-Validation.sh`. Expected: Each in-scope IWS shows **current** (service revision matches the deployed value); validation reports the IWS signal present. On QA/Prod, deployment was done with `iwsdeploy`, not the portal.

Test 6 — Review caught nothing unexpected

Do: Confirm the import review (Step 5.3) showed only the intended objects, and that no target-only object was unexpectedly replaced or left stale. Expected: The applied change set equals the intended scope; nothing outside the Migration Request changed.

Test 7 — Rollback point is verified (Production)

Do: For Production, confirm the pre-import backup restores and the prior bundle level is recorded. Expected: A verified restore path exists; the promotion is recoverable inside the change window.

Pass/fail gate

Gate	Requirement
QA gate	Tests 1–6 pass; configuration parity confirmed, code build matched, IWS current — and QA functional testing passed — before promoting to Production

Gate	Requirement
Production gate	Tests 1–6 pass for Production against QA, and Test 7 (verified rollback point) is satisfied inside the change window

6. Troubleshooting Guide — Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod)

Use `OC2MMIG-Discovery.sh` for environment/config inventory, `OC2MMIG-Validation.sh` for parity signals on a target, `OC2MMIG-Report.sh` for the source-vs-target compare, and `OC2MMIG-Remediation.sh` (dry-run by default) to preview corrective actions before applying. All four are read-only where they touch live systems, except Remediation which only acts when explicitly told to apply.

A. An object is missing in the target after import

Symptom	Cause	Remediation
A configuration object promoted fine in Dev but is absent in the target	No Migration Plan for that (often custom) object type, so it was silently excluded from the bundle	Author a Migration Plan for the object's MO/business object, add it to the Migration Request, re-export and re-import; confirm with `OC2MMIG-Report.sh`
An object in the Migration Request never appears in the export	The request's declared scope does not actually include it	Inspect the request with `OC2MMIG-ScopeHelper.sh`, correct the scope, re-export
A target-only object you expected to be removed is still present	CMA import **does not delete** target-only records	Retire the object deliberately in the target (import does not clean up); this is by design

B. The bundle will not export or import

Symptom	Cause	Remediation
Export produces no file	Migration Assistant Configuration missing/incomplete, or export directory not writable	Confirm the master record and directory ownership on the source; re-run `OC2MMIG-Validation.sh`
Import cannot find the file	File never reached the target import directory	Verify the transfer path (Requirement 2.5); copy the binary export file into the target import directory and retry

Symptom	Cause	Remediation
Import errors on apply	Source/target framework-version gap or unreconciled dependency	Compare levels with `OC2MMIG-Discovery.sh`; align target to source release/patch, or follow Oracle's cross-version CMA guidance (verify for your version on MOS)

C. Inbound Web Services are stale or failing

• The *Deployment Status* zone shows a service **not current** (its service revision differs from the

deployed value): the IWS metadata migrated with the bundle but the service was not (re)deployed. On the target, source `splenviron` and run `iwsdeploy[.sh]`, then re-check the status.

• IWS deployment "worked in Dev" but not in QA/Prod: the **online deployment portal is supported only in

non-production**. Use the command-line `iwsdeploy` for QA and Production.

• Build fails with no services deployed: at least one Inbound Web Service must be defined before the build

runs. Confirm the in-scope IWS exist in the target, then re-deploy.

D. Configuration is live but behaves wrong — code is out of step

• Symptom: the promoted configuration is present but logic behaves as it did before the change. Cause: the

code build/patch was not promoted in lockstep with the bundle. Run `OC2MMIG-Discovery.sh` on source and target and compare the build/patch level; install the matching build and re-run the post-install steps.

E. The import review showed unexpected changes

• CMA fully **replaces** matching objects from the source. If the review lists objects you did not intend

to change, **stop before applying** — your Migration Request scope is wider than the change. Inspect it with `OC2MMIG-ScopeHelper.sh`, narrow the request, re-export, and review again.

F. Compare report shows a gap after a "successful" import

• "Import OK" is not parity. Run `OC2MMIG-Report.sh` to see exactly which of config-objects / IWS-status /

build-level is out of parity, remediate that specific cause (re-import, re-deploy IWS, or re-apply the build), and re-compare. Use `OC2MMIG-Remediation.sh` to preview the corrective action first.

G. A promotion needs to be rolled back

• Restore the pre-import backup of the target, or re-import the previously retained bundle (recorded prior

bundle level), and re-deploy the prior IWS/build. This is why every import is preceded by a backup and every bundle is kept in version control. For Production, perform the rollback inside the same change window.

H. Diagnostics quick reference

```
./OC2MMIG-Discovery.sh   your-env       # release/patch level, CMA readiness, plans/requests present
./OC2MMIG-Validation.sh  your-target    # parity signals: import applied, IWS current, build matched
./OC2MMIG-Report.sh      your-source your-target   # source-vs-target compare for the promoted scope
./OC2MMIG-ScopeHelper.sh your-request   # print a Migration Request's declared scope
./OC2MMIG-Remediation.sh your-target    # dry-run by default; add --apply to act
```

I. When to escalate

If source and target are at the same level, the bundle imported without error, IWS deployment status reads current, and the build levels match — but the compare report still shows a gap after remediation — capture the `OC2MMIG-Discovery.sh`, `OC2MMIG-Validation.sh`, and `OC2MMIG-Report.sh` output and review the Migration Plans and Migration Request scope with the application-development and release teams. For CMA reference behavior or a suspected framework defect, raise it on My Oracle Support with the captured output (verify the note/patch for your version).

7. Best Practices — Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod)

• One named Migration Request per scope, reused every release. Build the request once and reuse it; ad-hoc

"export everything" bundles are neither repeatable nor auditable, and they promote things you did not intend to change.

• Check every bundle into version control. The binary export file is just a file — retain it so the exact

reviewed artifact that passed QA is the one applied to Production, and so every release is auditable.

• Promote configuration and code in lockstep. A CMA bundle without its matching code build (or vice versa)

is a half-promotion; record the bundle version, build/patch version, and IWS list together as one release and move them together.

• Author a Migration Plan for every custom object *before* building the request. An object type with no

plan is silently excluded from the bundle — the number-one cause of "it worked in QA but is missing in Prod."

• Always review before you apply. CMA fully replaces matching target objects and never deletes target-only

ones; the import review is your last chance to catch an unintended replacement — never click through it.

• Back up the target before every import. Because import replaces objects, the pre-import backup (plus the

recorded prior bundle level) is your rollback; take it every time, not just for Production.

• Deploy IWS from the command line for QA and Prod. The online deployment portal is supported only in

non-production; use `iwsdeploy` under `splenviron` for controlled environments and confirm the Deployment Status zone reads current.

• Compare, don't assume. Run the environment-compare step on every promotion; "the import said OK" is not

the same as "source and target are at parity." Parity confirmed is the evidence of a clean promotion.

• Promote one direction, through the gates. Dev → QA → Prod, never sideways or backward; pass the explicit

gate (parity + functional test) at each hop before the next.

• Prove it on a pilot scope first. Run one bounded Migration Request Dev → QA before promoting a full

release or touching Production — the pilot gate catches plan, transfer, and IWS gaps cheaply.

• Keep source and target at compatible levels. Same-release promotion is the safe default; treat a

framework-version gap as its own planned exercise, not an incidental part of a release.

• Never author configuration directly in QA or Prod. All change originates in Dev and is promoted; a

hand-edit in a downstream environment is exactly the drift this method exists to prevent.

8. Deliverables Summary — Oracle C2M Environment Migration & Code Promotion (Dev → QA → Prod)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The C2M promotion model: OUAF config migration (CMA/bundles), code/IWS promotion, and environment comparison (the why and the moving parts)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Oracle)	Category-wide prerequisites (entitlement, certified stack, WLST/shell/SQL, change control), published once
3. Implementation Guide	Exact, ordered steps to export/import the bundle, promote code, deploy IWS, compare, and validate — Dev → QA → Prod with gates

Document	Purpose
5. Testing Guide	Validation procedures with per-environment pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations for repeatable promotion

Named migration objects

Object	Role
Migration Assistant Configuration	System-wide CMA master record; enables export/import on each environment
Migration Plan	Defines what a record type looks like to CMA (MO + business object + related records); one per custom object type
Migration Request	The reusable unit of promotion — defines exactly which configuration a release bundles
Binary export file (bundle)	The self-contained CMA export, retained in version control and promoted onward

Automation scripts (shell / WLST / SQL — not PowerShell)

Script	What it does
`OC2MMIG-Discovery.sh`	Read-only inventory of C2M/OUAF release and patch level, CMA readiness, and Migration Plans/Requests present on an environment
`OC2MMIG-Validation.sh`	Read-only parity validation on a target: CMA import applied, IWS deployment current, code build level matched
`OC2MMIG-Report.sh`	Read-only source-vs-target compare report for a promoted scope (config objects, IWS status, build level)
`OC2MMIG-Remediation.sh`	Corrective helper for parity gaps; **dry-run by default**, prints intended actions before any apply
`OC2MMIG-ScopeHelper.sh`	Resolves and prints a Migration Request's declared scope so you promote exactly what you intend

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five automation scripts (shell / WLST / SQL) in a single ZIP.

• A repeatable promotion method following the model: Assess, then Configure, then Pilot, then Validate, then Expand.

Outcome

C2M configuration and code promoted cleanly from Dev to QA to Production — parity confirmed, no drift — so what you tested is exactly what goes live, pilot-first and rollback-ready.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OC2MMIG-Discovery.sh

Purpose: Inventory one Oracle Utilities C2M / OUAF environment so you can assess it before promoting: the C2M/OUAF release and patch/bundle level, whether the Configuration Migration Assistant (CMA) is ready (master config + export/import directories), and which Migration Plans and Migration Requests exist. Read-only — it inventories the environment, it never exports, imports, deploys, or changes anything. Run it against your-source-env and each your-target-env. HOW TO RUN source the product environment first, then: ./OC2MMIG-Discovery.sh your-env-name e.g. ./OC2MMIG-Discovery.sh QA Requires: product OS account with splenviron/splquery access and SQL*Plus (or SQLcl) to the C2M schema. Set the schema connect string in OC2MMIG_DB below or export OC2MMIG_DB. EXPECTED RESULTS A one-screen summary: release/patch level, a CMA-ready PASS/GAP line (master config + directories), and counts of Migration Plans and Migration Requests present. Record these against the Requirements checklist. GAP lines are what you remediate before configuring. NOTES White-label, the repeatable way. Read-only against a live system. Object/column names follow OUAF conventions; verify names for your C2M version (C2M 25.4 / min v2.9). MOS-gated reference detail is pointed to, never reproduced.

How to run: source the product environment first, then: ./OC2MMIG-Discovery.sh your-env-name e.g. ./OC2MMIG-Discovery.sh QA Requires: product OS account with splenviron/splquery access and SQL*Plus (or SQLcl) to the C2M schema. Set the schema connect string in OC2MMIG_DB below or export OC2MMIG_DB. EXPECTED RESULTS A one-screen summary: release/patch level, a CMA-ready PASS/GAP line (master config + directories), and counts of Migration Plans and Migration Requests present. Record these against the Requirements checklist. GAP lines are what you remediate before configuring. NOTES White-label, the repeatable way. Read-only against a live system. Object/column names follow OUAF conventions; verify names for your C2M version (C2M 25.4 / min v2.9). MOS-gated reference detail is pointed to, never reproduced.

OC2MMIG-Remediation.sh

Purpose: Close the common parity gaps a compare turns up on a target: re-import the retained CMA bundle, re-deploy an in-scope Inbound Web Service with iwsdeploy, or re-run the code post-install for the promoted build. DRY-RUN by default — it prints the exact command it would run and changes nothing until you pass --apply. Even with --apply it acts only on the target you name and the bundle/IWS you name; it never invents scope. HOW TO RUN dry-run (default, safe) prints the plan: ./OC2MMIG-Remediation.sh your-target-env --bundle your-bundle-file --iws your-iws-app to actually perform it, add --apply (source splenviron first; run in the change window): ./OC2MMIG-Remediation.sh your-target-env --bundle your-bundle-file --iws your-iws-app --apply EXPECTED RESULTS Dry-run: a numbered plan of the exact import/iwsdeploy/post-install commands, then "DRY-RUN — nothing changed." With --apply: each step runs and reports success/failure; stop on the first failure and re-compare with OC2MMIG-Report before continuing. NOTES White-label, the repeatable way. DRY-RUN by default; --apply is the only way it mutates, and only inside a change window with a verified backup. QA/Prod IWS deployment is command-line iwsdeploy (the portal is dev-only). Verify utility names/switches for your C2M version.

How to run: dry-run (default, safe) prints the plan: ./OC2MMIG-Remediation.sh your-target-env --bundle your-bundle-file --iws your-iws-app to actually perform it, add --apply (source splenviron first; run in the change window): ./OC2MMIG-Remediation.sh your-target-env --bundle your-bundle-file --iws your-iws-app

--apply EXPECTED RESULTS Dry-run: a numbered plan of the exact import/iwsdeploy/post-install commands, then "DRY-RUN — nothing changed." With --apply: each step runs and reports success/failure; stop on the first failure and re-compare with OC2MMIG-Report before continuing. NOTES White-label, the repeatable way. DRY-RUN by default; --apply is the only way it mutates, and only inside a change window with a verified backup. QA/Prod IWS deployment is command-line iwsdeploy (the portal is dev-only). Verify utility names/switches for your C2M version.

OC2MMIG-Report.sql

Purpose: Produce the environment-comparison evidence that a promotion reached parity. Run once against the source schema and once against each target schema, then diff the two outputs: it lists, for the objects in a Migration Request, each object's key and last-change stamp, plus the configuration release/bundle level. Matching output across environments = parity for the scope. Read-only — SELECT only, it changes nothing. HOW TO RUN run against source, then against target, capturing each to a file, then compare: sqlplus -s cisadm@source @OC2MMIG-Report.sql your-request-code > report_source.txt sqlplus -s cisadm@target @OC2MMIG-Report.sql your-request-code > report_target.txt diff report_source.txt report_target.txt Pass the Migration Request code as the first argument (defaults to a placeholder). EXPECTED RESULTS A stable, sorted listing (object type, key, version/last-changed) for the request's scope and the config bundle level. An empty diff between source and target = PASS (parity, no drift). Any diff names exactly which object is missing or mismatched on the target. NOTES White-label, the repeatable way. Read-only. Table/column names follow OUAF conventions; verify names for your C2M version (C2M 25.4 / min v2.9). Detailed CMA reference behavior is on MOS — pointed to, not reproduced here.

How to run: run against source, then against target, capturing each to a file, then compare: sqlplus -s cisadm@source @OC2MMIG-Report.sql your-request-code > report_source.txt sqlplus -s cisadm@target @OC2MMIG-Report.sql your-request-code > report_target.txt diff report_source.txt report_target.txt Pass the Migration Request code as the first argument (defaults to a placeholder). EXPECTED RESULTS A stable, sorted listing (object type, key, version/last-changed) for the request's scope and the config bundle level. An empty diff between source and target = PASS (parity, no drift). Any diff names exactly which object is missing or mismatched on the target. NOTES White-label, the repeatable way. Read-only. Table/column names follow OUAF conventions; verify names for your C2M version (C2M 25.4 / min v2.9). Detailed CMA reference behavior is on MOS — pointed to, not reproduced here.

OC2MMIG-ScopeHelper.sh

Purpose: Show exactly what a Migration Request will bundle before you export it, so you promote what you intend and nothing more. It prints the request's selectors — the Migration Plans and object keys it pulls — and flags any in-scope object type that has no Migration Plan (which CMA would silently drop). Read-only — it inspects the request definition, it never exports or imports. HOW TO RUN source the product environment, then: ./OC2MMIG-ScopeHelper.sh your-request-code e.g. ./OC2MMIG-ScopeHelper.sh your-rate-config-request Set the schema connect string in OC2MMIG_DB below or export OC2MMIG_DB. EXPECTED RESULTS A readable listing of the request's declared scope (plan, object key) and a PASS/GAP line: PASS if every selected object type has a Migration Plan, GAP (with the object named) if one does not — author the missing plan before exporting, or the object will be left out. NOTES White-label, the repeatable way. Read-only against a live system. The Migration Request is the reusable unit of promotion — build once, reuse every release. Table/column names follow OUAF conventions; verify names for your C2M version (C2M 25.4 / min v2.9).

How to run: source the product environment, then: ./OC2MMIG-ScopeHelper.sh your-request-code e.g. ./OC2MMIG-ScopeHelper.sh your-rate-config-request Set the schema connect string in OC2MMIG_DB below or export OC2MMIG_DB. EXPECTED RESULTS A readable listing of the request's declared scope (plan, object key) and a PASS/GAP line: PASS if every selected object type has a Migration Plan, GAP (with the object named) if one does not — author the missing plan before exporting, or the object will be left out. NOTES White-label, the repeatable way. Read-only against a live system. The Migration Request is the reusable unit of promotion — build once, reuse every release. Table/column names follow OUAF conventions; verify names for your C2M version (C2M 25.4 / min v2.9).

OC2MMIG-Validation.py

Purpose: Confirm a C2M/OUAF promotion actually landed on a target environment, from the WebLogic side: that the Inbound Web Service (IWS) modules for the release are deployed and in the active state on the C2M managed server, and that the domain is reachable. Pair it with the SQL import/build checks from OC2MMIG-Report to get the full parity picture. Read-only — it reads deployment state through WLST, it never deploys, redeploys, starts, or stops anything. HOW TO RUN source the product environment, then run under the OUAF WLST wrapper, e.g. wlst.sh OC2MMIG-Validation.py your-target-env Connection comes from the environment (ENVIRON.INI / wlst connect), or export: OC2MMIG_ADMIN_URL e.g. t3://your-admin-host:7001 OC2MMIG_WLS_USER WebLogic admin user (read-only monitor role is sufficient) The password is read from OC2MMIG_WLS_PASS if set, otherwise WLST prompts (never hard-coded). EXPECTED RESULTS A PASS/GAP line per in-scope IWS module (STATE = active vs not-deployed/stale) and a domain reachability line. All PASS means the IWS side of parity is met; any GAP names the module to (re)deploy with iwsdeploy on the target. Non-zero exit if any GAP is found. NOTES White-label, the repeatable way. Read-only against a live domain. WLST/Jython, not PowerShell. The online IWS deployment portal is dev-only; QA/Prod use command-line iwsdeploy. Verify MBean paths/service names for your WebLogic 14c / C2M version.

How to run: source the product environment, then run under the OUAF WLST wrapper, e.g. wlst.sh OC2MMIG-Validation.py your-target-env Connection comes from the environment (ENVIRON.INI / wlst connect), or export: OC2MMIG_ADMIN_URL e.g. t3://your-admin-host:7001 OC2MMIG_WLS_USER WebLogic admin user (read-only monitor role is sufficient) The password is read from OC2MMIG_WLS_PASS if set, otherwise WLST prompts (never hard-coded). EXPECTED RESULTS A PASS/GAP line per in-scope IWS module (STATE = active vs not-deployed/stale) and a domain reachability line. All PASS means the IWS side of parity is met; any GAP names the module to (re)deploy with iwsdeploy on the target. Non-zero exit if any GAP is found. NOTES White-label, the repeatable way. Read-only against a live domain. WLST/Jython, not PowerShell. The online IWS deployment portal is dev-only; QA/Prod use command-line iwsdeploy. Verify MBean paths/service names for your WebLogic 14c / C2M version.